4
Text Classification

This chapter builds on a brief introduction to text classification and provides you with an example of the **NaÃ¯ve Bayes** algorithm, developed from scratch in order to explain how to turn an equation into code.

In this chapter, we will cover:

- Learning and classification
- Bayesian classification
- NaÃ¯ve Bayes algorithm
- E-mail subject line tester
- The data
- The algorithm
- Classifier accuracy

Learning and classification

When we want to automatically identify which category belongs to a specific value (categorical value), we need to implement an algorithm that can decide the most likely category for the value based on previous data. This is called a **classifier**. In the words of Tom Mitchell:

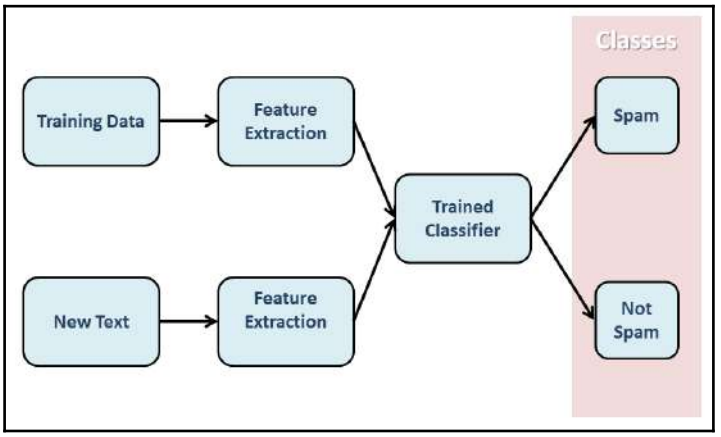
> *"How can we build computer systems that automatically improve with experience, and what are the fundamental laws that govern all learning processes?"*

2016. Packt Publishing.
All rights reserved. May not be reproduced in any form without permission from the publisher, except fair uses permitted under U.S. or applicable copyright law.

EBSCO Publishing: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD 1364690; Hector Cuesta, Dr. Sampath Kumar; Practical Data Analysis - Second Edition Account:ehost.

The key word here is learning (supervised learning, in this case) and knowing how to train an algorithm to identify categorical elements. The common examples are spam classification, speech recognition, search engines, computer vision, and language detection, but there is a large number of applications for a classifier. We can find two kinds of problems in classification. The **Binary classification** is where we only have two categories (**Spam** or **Not Spam**) and the **Multiclass classification**, in which there are many categories involved (such as the opinions Positive, Neutral, Negative, and so on). We can find several algorithms for classification; the most frequently used are support vector machines, neural networks, decision trees, NaÃ¯ve Bayes, and hidden Markov models. In this chapter, we will implement a probabilistic classification through the NaÃ¯ve Bayes algorithm, but in the following chapters, we will implement several other classification algorithms for a variety of problems.

The general steps involved in supervised classification are shown in the following diagram. First, we will collect training data (previously classified), then we will perform feature extraction (relevant features for the categorization), and next we will train the algorithm with the features vector. Once we get our trained classifier, we can insert new strings, extract its features, and send them to the classifier. Finally, the classifier will give us the most likely class (category) for the new string.



Additionally, we will test the classifier's accuracy by using a hand-classified test set. Due to this, we will split the data into two sets: the training data and the test data.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Bayesian classification

The probabilistic classification is a practical way to perform inferences based on data using statistical inferences to find the best class for a given value. Given a probability distribution, we can select the best option with the highest probability. The **Bayes Theorem** is the basic rule to perform inferences. The theorem allows us to update the likelihood of an event given the new data or observations. In other words, it allows us to update the prior probability *P (A)* to the posterior probability *P (A|B)*. The prior probability is given by the likelihood before the data is evaluated and the posterior probability is assigned after the data is taken into account. The following expression represents the Bayes Theorem:

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)}$$

$P(A \mid B)$ =The conditional probability of **A** given **B**

NaÃ¯ve Bayes

NaÃ¯ve Bayes is the simplest classification algorithm among Bayesian classification methods. In this algorithm, we simply need to learn the probabilities by making the assumption that the attributes *A* and *B* are independents, hence the reason this model is defined as an *independent feature model*. NaÃ¯ve Bayes is widely used in text classification because the algorithm can be trained easily and efficiently. In NaÃ¯ve Bayes, we can calculate the probability of a condition *A* if *B (P(A|B))*, if you already know the probability of *B* given *A (P(B|A))*, as well as to *A (P(A))* and *B (P(B))* individually, as is shown in the previous Bayes Theorem example.

E-mail subject line tester

Spam is junk e-mail, which is understood as **Unsolicited Bulk Email** (**UBE**). E-mails can be blocked before they are delivered to the recipient based on e-mail filter reports. The e-mail filter scans the subject line of e-mails for spam or ham (e-mail that is not spam is often called ham). One of the e-mail filters is the e-mail subject line filter. Over 35 percent of spam mails are detected from the subject line of an e-mail.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

An **E-mail Subject Line Tester** is a simple program that will define whether a certain subject line in an e-mail is spam or not. In this chapter, we will program a NaÃ¯ve Bayes classifier from scratch. The example will classify whether a subject line is a spam or not with a very simple code by breaking the subject lines into a list of relevant words that will be used as a feature vector in the algorithm. In order to do this, we will use the **SpamAssassin** public dataset. SpamAssasin includes three categories: **spam**, **easy ham**, and **hard ham**. In this case, we shall only use *spam* and *easy ham* (not spam) to train and test the classifier as we code a binary classifier.

There are several features that we can use for our classifiers such as the precedence, language, and the use of upper case. We will keep things simple and use the frequency of only words with more than three characters, and will avoid words such as *the* or *RT:* when training the algorithm.

We will implement the Bayes Rule using the words and categories, as shown in the following equation:

$$P(word \mid category) = \frac{P(category \mid word)P(word)}{P(category)}$$

For more information about probability distributions, please refer to:
`http://en.wikipedia.org/wiki/Probability_distribution`

We have two classes in the categories that represent whether a Subject Line is *Spam* or *Not Spam*. We need to split the texts into a list of words in order to get the likelihood of each word. Once we know the probability of each word, we need to multiply the probabilities for each category, as shown in the following equation:

$$P(category \mid word_1, word_2, ..., word_n) = P(category) x \Pi_i P(word \mid category)$$

In other words, we multiply the likelihood of each word *P(word|category)* of the subject line and the probability of the category *P(category)*.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

For training the algorithm, we need to provide some prior examples. In this case, we use the function `training()`, which needs a dictionary of subject line and category, as we can see in the following examples:

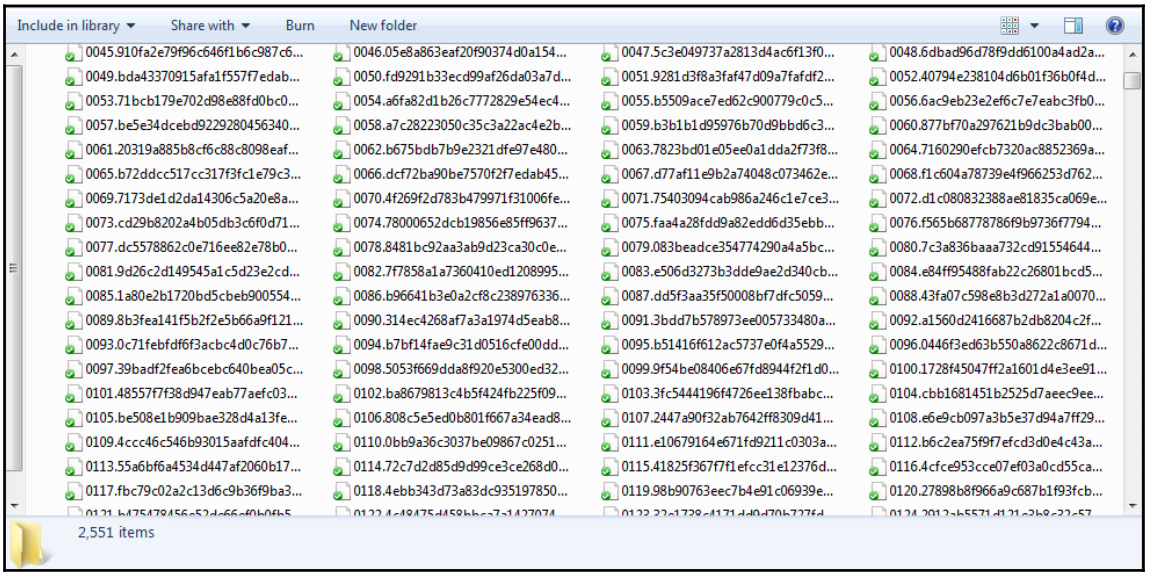
```
Re: Tiny DNS Swap, nospam
Save up to 70% on international calls!, nospam
[Ximian Updates] Hyperlink handling in Gaim allows arbitrary code to be
executed, nospam
Promises., nospam
Life Insurance - Why Pay More?, spam
[ILUG] Guaranteed to lose 10-12 lbs in 30 days 10.206, spam
.  .  .
```

The data

We can find the spam dataset from the following link:

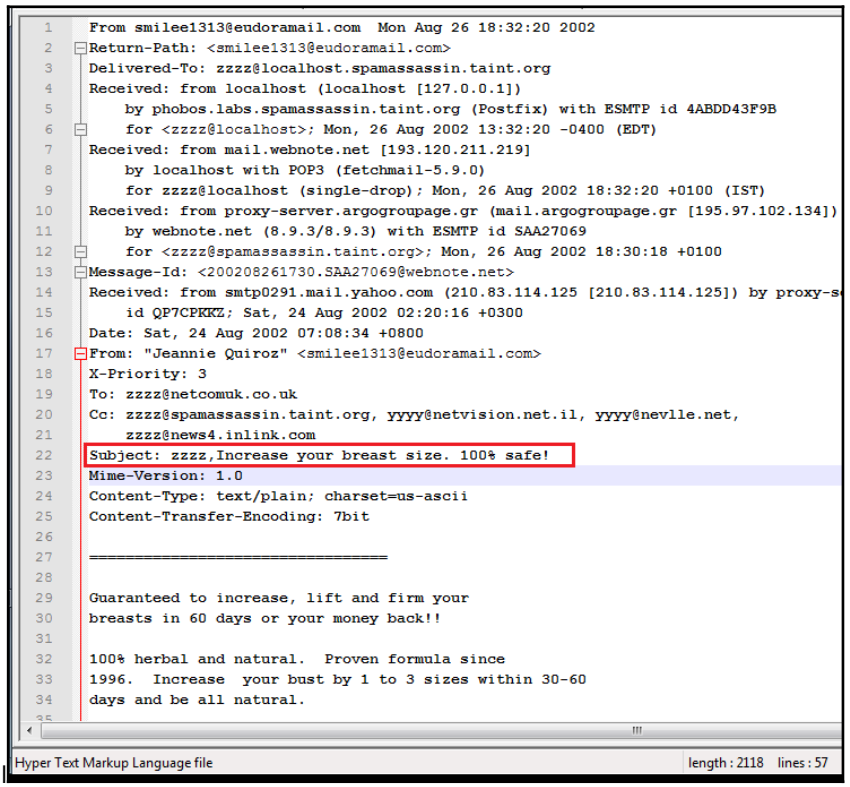
`http://spamassassin.apache.org/`

In the following screenshot, we can see the *easy ham* (not spam) folder with 2551 files:



EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

The spam text looks like the following screenshot, which may include HTML tags and plain text. In this case, we are only interested in the subject line, so we need to write a code to obtain the subject from all the files.



```
From smilee1313@eudoramail.com  Mon Aug 26 18:32:20 2002
Return-Path: <smilee1313@eudoramail.com>
Delivered-To: zzzz@localhost.spamassassin.taint.org
Received: from localhost (localhost [127.0.0.1])
    by phobos.labs.spamassassin.taint.org (Postfix) with ESMTP id 4ABDD43F9B
    for <zzzz@localhost>; Mon, 26 Aug 2002 13:32:20 -0400 (EDT)
Received: from mail.webnote.net [193.120.211.219]
    by localhost with POP3 (fetchmail-5.9.0)
    for zzzz@localhost (single-drop); Mon, 26 Aug 2002 18:32:20 +0100 (IST)
Received: from proxy-server.argogroupage.gr (mail.argogroupage.gr [195.97.102.134])
    by webnote.net (8.9.3/8.9.3) with ESMTP id SAA27069
    for <zzzz@spamassassin.taint.org>; Mon, 26 Aug 2002 18:30:18 +0100
Message-Id: <200208261730.SAA27069@webnote.net>
Received: from smtp0291.mail.yahoo.com (210.83.114.125 [210.83.114.125]) by proxy-s
    id QP7CPKKZ; Sat, 24 Aug 2002 02:20:16 +0300
Date: Sat, 24 Aug 2002 07:08:34 +0800
From: "Jeannie Quiroz" <smilee1313@eudoramail.com>
X-Priority: 3
To: zzzz@netcomuk.co.uk
Cc: zzzz@spamassassin.taint.org, yyyy@netvision.net.il, yyyy@nevlle.net,
    zzzz@news4.inlink.com
Subject: zzzz,Increase your breast size. 100% safe!
Mime-Version: 1.0
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

================================

Guaranteed to increase, lift and firm your
breasts in 60 days or your money back!!

100% herbal and natural.  Proven formula since
1996.  Increase  your bust by 1 to 3 sizes within 30-60
days and be all natural.
```

Hyper Text Markup Language file length : 2118 lines : 57

This example will show you how to preprocess the SpamAssassin data using Python in order to collect all the subject lines from the e-mails.

First, we need to import the `os` module in order to get the list of file names using the `listdir` function from the `" \spam"` and `" \easy_ham"` folders:

```
import os
files = os.listdir(r" \spam")
```

Now we need a new file to store the subject lines and the category (spam or not spam); this time, we will use a comma as a separator:

```
with open("SubjectsSpam.out","a") as out:
     category = "spam"
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

Now we will parse each file and get the subject. Finally, we write the subject and the category in the new file and delete all the commas from the subject lines `(line.replace(",", ""))` to skip future troubles with the CSV format:

```
for fname in files:
   with open("\\spam" + fname) as f:
       data = f.readlines()
       for line in  data:
           if line.startswith("Subject:"):
               line.replace(",", "")
               print(line)
               out.write("{0}, {1} \n".format(line[8:-1], category))
```

We need to clean up the text, skipping the word *subject:* (the first eight characters) and the enter at the end of the text; in order to do this, we will use line[8:-1].

```
Output:
>>>Hosting from ?6.50 per month
>>>Want to go on a date?
>>>[ILUG] ilug,Bigger, Fuller Breasts Naturally In Just Weeks
>>> zzzz Increase your breast size. 100% safe!
. . .
```

We will keep the spam and not spam in different files to play with the size of the training and test sets. Usually, more data in the training set means better algorithm performance, but in this case, we will try to find an optimal threshold of the training set size.

All the codes and datasets of this chapter can be found in the author's GitHub repository at:
`https://github.com/hmcuesta/PDA_Book/tree/master/Chapter4`

The algorithm

We use the function `list_words()` to get a list of unique words with more than three characters in lower case:

```
def list_words(text):
    words = []
    words_tmp = text.lower().split()
    for w in words_tmp:
        if w not in words and len(w) > 3:
            words.append(w)
    return words
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

For a more advanced term-document matrix, we can use the Python textmining package from:
https://pypi.python.org/pypi/textmining/1.

The `training()` function creates variables to store the data needed for the classification. The `c_words` variable is a dictionary with the unique words and its number of occurrences in the text (frequency) by category. The `c_categories` variable stores a dictionary of each category and its number of texts. Finally, `c_text` and `c_total_words` store the total count of texts and words, respectively:

```
def training(texts):
    c_words ={}
    c_categories ={}
    c_texts = 0
    c_total_words =0
    #add the classes to the categories
    for t in texts:
        c_texts = c_texts + 1
        if t[1] not in c_categories:
            c_categories[t[1]] = 1
        else:
            c_categories[t[1]]= c_categories[t[1]] + 1

    #add the words with list_words() function
    for t in texts:
        words = list_words(t[0])

    for p in words:
        if p not in c_words:
            c_total_words = c_total_words +1
            c_words[p] = {}
            for c in c_categories:
                c_words[p][c] = 0
        c_words[p][t[1]] = c_words[p][t[1]] + 1

    return (c_words, c_categories, c_texts, c_total_words)
```

The `classifier()` function applies the Bayes rule and classifies the subject into one of the two categories, that is, either spam or not spam. The function also needs the four variables from the `training()` function:

```
def classifier(subject_line, c_words, c_categories, c_texts, c_tot_words):
    category =""
    category_prob = 0
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
    for c in c_categories:
        #category probability
        prob_c = float(c_categories[c])/float(c_texts)
        words = list_words(subject_line)
        prob_total_c = prob_c
        for p in words:
            #word probability
            if p in c_words:
                prob_p= float(c_words[p][c])/float(c_tot_words)
                #probability P(category|word)
                prob_cond = prob_p/prob_c
                #probability P(word|category)
                prob =(prob_cond * prob_p)/ prob_c
                prob_total_c = prob_total_c * prob

            if category_prob < prob_total_c:
                category = c
                category_prob = prob_total_c
    return (category, category_prob)
```

Finally, we will read the `training.csv` file, which contains the training dataset; in this case, 100 spam and 100 not spam subject lines:

```
if __name__ == "__main__":
    with open('training.csv') as f:
        subjects = dict(csv.reader(f, delimiter=','))
    words,categories,texts,total_words = training(subjects)
```

Now, to check if everything is working correctly, we test the classifier with one subject line:

```
    clase = classifier("Low Cost Easy to Use Conferencing"
                       , words,categories,texts,total_words)

    print("Result: {0} ".format(clase))
```

We can see the result in the Python console, and from the result we can see that the classifier is, so far, working correctly:

```
    >>> Result: ('spam', 0.18518518518518517)
```

We can see the complete code of the NaÃ¯ve Bayes classifier listed here:

```
import csv
def list_words(text):
    words = []
    words_tmp = text.lower().split()
    for p in words_tmp:
        if p not in words and len(p) > 3:
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
            words.append(p)
    return words

def training(texts):
    c_words ={}
    c_categories ={}
    c_texts = 0
    c_tot_words =0
    for t in texts:
        c_texts = c_texts + 1
        if t[1] not in c_categories:
            c_categories[t[1]] = 1
        else:
            c_categories[t[1]]= c_categories[t[1]] + 1

    for t in texts:
        words = list_words(t[0])

    for p in words:
        if p not in c_words:
            c_tot_words = c_tot_words +1
            c_words[p] = {}
            for c in c_categories:
                c_words[p][c] = 0
        c_words[p][t[1]] = c_words[p][t[1]] + 1

    return (c_words, c_categories, c_texts, c_tot_words)


def classifier(subject_line, c_words, c_categories, c_texts, c_tot_words):
    category =""
    category_prob = 0

    for c in c_categories:
        prob_c = float(c_categories[c])/float(c_texts)
        words = list_words(subject_line)
        prob_total_c = prob_c
        for p in c_words:
            if p in words:
                prob_p= float(c_words[p][c])/float(c_tot_words)
                prob_cond = prob_p/prob_c
                prob =(prob_cond * prob_p)/ prob_c
                prob_total_c = prob_total_c * prob

            if category_prob < prob_total_c:
                category = c
                category_prob = prob_total_c
    return (category, category_prob)
```


EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

```
if __name__ == "__main__":

    with open('training.csv') as f:
         subjects = dict(csv.reader(f, delimiter=','))

    w,c,t,tw = training(subjects)
    clase = classifier("Low Cost Easy to Use Conferencing"
                       ,w,c,t,tw)
    print("Result: {0} ".format(clase))
```

Finally, we will execute the classifier against the test set in order to prove accuracy. We will test 100 new records and get a new number of cases classified correctly.

```
with open("test.csv") as f:
   correct = 0
   tests = csv.reader(f)
   for subject in test:
        clase = classifier(subject[0],w,c,t,tw)
        if clase[1] =subject[1]:
      correct += 1
    print("Efficiency : {0} of 100".format(correct))
```

Classifier accuracy

Now we need to test our classifier with a bigger test set; in this case, we will randomly select 100 subjects: 50 spam and 50 not spam. Finally, we will count how many times the classifier chose the correct category:

```
with open("test.csv") as f:
    correct = 0
    tests = csv.reader(f)
    for subject in test:
          clase = classifier(subject[0],w,c,t,tw)
          if clase[1] =subject[1]:
      correct += 1
     print("Efficiency : {0} of 100".format(correct))
```

In this case, the `Efficiency` is `82` percent:

```
>>> Efficiency: 82 of 100
```

We can use an out of the box implementation of the Naive Bayes classifier, like the `NaiveBayesClassifier` function in the NLTK package for Python. NLTK provides a very powerful natural language toolkit and we can download it from `http://nltk.org/`.

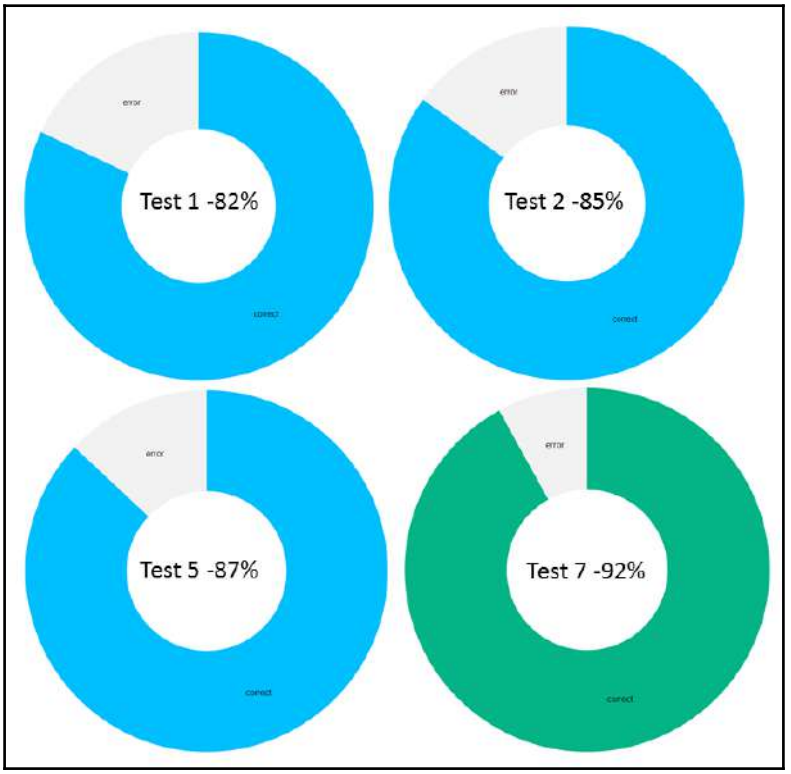
EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

In `Chapter 1`, *Getting Started*, we presented a more sophisticated version of the NaÃ¯ve Bayes classifier to perform a sentiment analysis.

In this case, we will find an optimal size threshold for the training set. We will try a different number of random subject lines. In the following figure, we can see four of the seven tests and the classification rate (accuracy) of the algorithm. In all cases, we use the same test set of 100 elements and we use the same number of Email Subject Lines for each category:

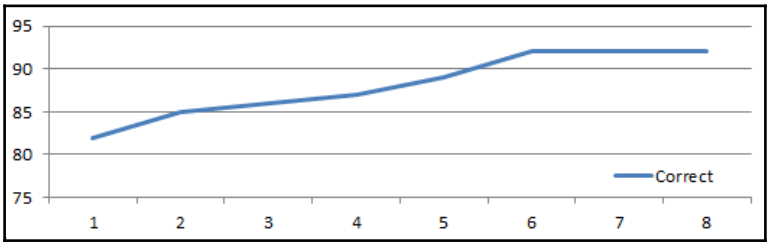
- Test 1: 82 percent with a training set of 200 elements
- Test 2: 85 percent with a training set of 300 elements
- Test 5: 87 percent with a training set of 500 elements
- Test 7: 92 percent with a training set of 800 elements

We run several trials with a different number of records to train the classifier, and we can see that the accuracy is improved with a bigger training set:



EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.

As we can see in the following diagram, for this specific example, the maximum accuracy is **92** percent and the optimal number of texts in the training set is 700. After 700 texts in the training set, the accuracy of the classifier doesn't see a significant improvement:



Summary

In this chapter, we created a basic but useful e-mail subject line tester. This chapter provides a guide on how to code a basic NaÃ¯ve Bayes classifier from scratch, without any external library, in order to demonstrate how easy it is to program a machine learning algorithm. We also defined a maximum size threshold for the training set and got an accuracy of 92 percent, which for this basic example is quite good.

In the following chapters, we will introduce more complex machine learning algorithms using the mlpy library and we will present how to extract more sophisticated features.

EBSCOhost: eBook Collection (EBSCOhost) printed on 3/13/2026 7:25:47 PM UTC via TORRENS GLOBAL EDUCATION SERVICES PTY LTD. All use subject to https://www.ebsco.com/terms-of-use.